

1. Function Basics

- Defining a function with `def`
- Calling a function
- Function naming
- Function body
- Indentation in functions
- Functions with no parameters
- Functions with no return value
- Calling a function multiple times
- Function execution flow

2. Parameters

- Parameters vs arguments
- One parameter
- Multiple parameters
- Passing different values
- Positional arguments
- Parameter order
- Parameters with different data types
- Passing expressions as arguments

3. Return Values

- `return`
- Returning one value
- Returning multiple values
- Storing returned values
- Using returned values in expressions
- Returning from conditional logic
- `return` vs `print()`
- Early `return`

4. Default Arguments

- Default parameter values
- Calling without optional arguments
- Overriding default values
- Multiple default parameters
- Required vs optional parameters
- Rules for placing default parameters

5. Keyword Arguments

- Passing arguments by parameter name
- Positional vs keyword arguments
- Mixing positional and keyword arguments
- Changing argument order with keywords
- Keyword arguments with default values
- Common mistakes with keyword arguments

6. Variable Scope

- Local variables
- Global variables
- Local vs global scope
- Accessing global variables
- `global` keyword
- Scope inside nested functions
- Variable lifetime
- Name resolution basics

7. Lambda Functions

- Creating a lambda
- Lambda parameters
- Lambda return value
- Calling lambda functions
- Lambda with one parameter
- Lambda with multiple parameters
- Lambda with `map()`
- Lambda with `filter()`
- Lambda with `sorted()`
- Lambda vs normal function

8. Recursion Basics

- Function calling itself
- Base case
- Recursive case
- Recursion flow
- Simple counting recursion
- Factorial using recursion
- Sum using recursion
- String recursion
- List recursion
- Recursion vs loop
- Common recursion mistakes

9. Modules

- What a module is
- Python's built-in modules
- Using a module
- Accessing module members
- Module aliases
- Importing specific members
- Module namespace
- `__name__`
- `if __name__ == "__main__":`

10. Import Statements

- `import module`
- `import module as alias`
- `from module import item`
- `from module import item1, item2`
- `from module import *`
- Importing functions
- Importing variables
- Importing classes
- Multiple imports
- Import errors

11. Packages

- Package structure
- Creating a package
- Submodules
- `__init__.py`
- Importing from a package
- Importing submodules
- Package aliases
- Nested packages
- Package vs module

12. Creating Custom Modules

- Creating your own `.py` module
- Writing functions inside a module
- Importing your custom module
- Using custom module functions
- Custom module variables
- Custom module constants
- Module aliases
- Creating multiple custom modules
- Organizing modules
- Creating a small custom package

13. Practical Integration

- Calculator using functions
- Student grade calculator
- Temperature converter
- Simple banking functions
- Function-based menu program
- Recursive number problems
- Lambda-based data processing
- Custom utility module
- Multi-module Python project
- Small package-based project